

无线CarPlay Wifi 连接流程详解

从无线 CarPlay 连接整个流程角度来说，相对于前半部分的蓝牙传输部分，iap2 on bluetooth 这个部分涉及到传输等等都是明文的方式，可以直接抓取蓝牙的hci日志可以清楚的知道无线 CarPlay 传输了什么报文以及交互流程，但是一旦过渡到 wifi 部分，这个部分我们可以抓取无线 wifi 信号或者也可以抓取经过网卡驱动后的以太网报文。

这里以抓取热点网卡的 pcap 为起点，后续的分析都将基于热点驱动处理后以太网帧报文，

下面就是抓取 7000 端口的所有报文：

1	0.000000	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:f603:c276	TCP	100	49224 → 7000 [ACK] Seq=0 Min=5535 Len=0 MSS=1440 USe=64 Tsv=1-2799007397 TSecr=0 SACK_PERM
2	0.000224	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:f603:c276	TCP	492	49224 → 7000 [ACK] Seq=1451818 Min=5535 Len=0 MSS=1440 USe=64 Tsv=1-2799007397 TSecr=0 SACK_PERM
3	0.004679	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:f603:c276	TCP	88	49224 → 7000 [ACK] Seq=1 Ack=1 Wln=13182 Len=0 Tsv=1-2799007405 TSecr=2063912586
4	0.009118	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:f603:c276	TCP	364	49224 → 7000 [PSH, ACK] Seq=1 Ack=1 Wln=13182 Len=276 Tsv=1-2799007411 TSecr=2063912586
5	0.009324	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:f603:c276	TCP	88	7000 → 49224 [ACK] Seq=1 Ack=277 Wln=64000 Len=0 Tsv=1-2063912589 TSecr=2799007411
6	0.012237	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:2904:e04c	TCP	360	7000 → 49224 [PSH, ACK] Seq=1 Ack=277 Wln=64512 Len=272 Tsv=1-2063912572 TSecr=2799007411 [TCP PDU reassemble
7	0.016290	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:f603:c276	TCP	88	49224 → 7000 [ACK] Seq=277 Ack=273 Wln=131136 Len=0 Tsv=1-2799007418 TSecr=2063912572
8	0.025379	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:f603:c276	TCP	453	49224 → 7000 [PSH, ACK] Seq=277 Ack=273 Wln=131136 Len=385 Tsv=1-2799007422 TSecr=2063912572
9	0.025447	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:2904:e04c	TCP	88	7000 → 49224 [ACK] Seq=273 Ack=442 Wln=64512 Len=0 Tsv=1-2063912585 TSecr=2799007422
10	0.030302	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:f603:c276	TCP	202	7000 → 49224 [PSH, ACK] Seq=273 Ack=442 Wln=64512 Len=114 Tsv=1-2063912590 TSecr=2799007422 [TCP PDU reassemb
11	0.032071	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:f603:c276	TCP	88	49224 → 7000 [ACK] Seq=442 Ack=387 Wln=131872 Len=0 Tsv=1-2799007435 TSecr=2063912590
12	0.036120	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:f603:c276	TCP	266	49224 → 7000 [PSH, ACK] Seq=442 Ack=387 Wln=131872 Len=178 Tsv=1-2799007437 TSecr=2063912590
13	0.036174	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:2904:e04c	TCP	88	7000 → 49224 [ACK] Seq=387 Ack=820 Wln=64512 Len=0 Tsv=1-2063912596 TSecr=2799007437
14	0.040879	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:f603:c276	TCP	912	7000 → 49224 [PSH, ACK] Seq=387 Ack=820 Wln=64512 Len=843 Tsv=1-2063912607 TSecr=2799007437 [TCP PDU reassemb
15	0.535874	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:f603:c276	TCP	88	49224 → 7000 [ACK] Seq=820 Ack=1238 Wln=130240 Len=0 Tsv=1-2799007397 TSecr=2063913047
16	0.551156	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:f603:c276	TCP	1086	49224 → 7000 [PSH, ACK] Seq=820 Ack=1238 Wln=131872 Len=908 Tsv=1-2799007590 TSecr=2063913047
17	0.551276	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:2904:e04c	TCP	88	7000 → 49224 [ACK] Seq=1238 Ack=1818 Wln=64000 Len=0 Tsv=1-2063913111 TSecr=2799007590
18	0.570400	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:f603:c276	TCP	500	7000 → 49224 [PSH, ACK] Seq=1238 Ack=1818 Wln=64512 Len=412 Tsv=1-2063913118 TSecr=2799007590 [TCP PDU reassemb
19	0.580952	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:f603:c276	TCP	88	49224 → 7000 [ACK] Seq=1818 Ack=1642 Wln=130608 Len=0 Tsv=1-2799007605 TSecr=2063913118
20	0.602662	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:f603:c276	TCP	588	49224 → 7000 [PSH, ACK] Seq=1818 Ack=1642 Wln=131872 Len=508 Tsv=1-2799008005 TSecr=2063913118
21	0.614999	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:2904:e04c	TCP	1136	7000 → 49224 [PSH, ACK] Seq=1642 Ack=2318 Wln=64512 Len=1042 Tsv=1-2063913175 TSecr=2799008005 [TCP PDU reassemb
22	0.615173	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:2904:e04c	TCP	1136	7000 → 49224 [ACK] Seq=2318 Ack=2318 Wln=64512 Len=1042 Tsv=1-2063913175 TSecr=2799008005 [TCP PDU reassemb
23	0.615272	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:2904:e04c	TCP	1136	7000 → 49224 [PSH, ACK] Seq=2726 Ack=2318 Wln=64512 Len=1042 Tsv=1-2063913175 TSecr=2799008005 [TCP PDU reassemb
24	0.615365	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:2904:e04c	TCP	1136	7000 → 49224 [ACK] Seq=2726 Ack=2318 Wln=64512 Len=1042 Tsv=1-2063913175 TSecr=2799008005 [TCP PDU reassemb
25	0.615457	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:2904:e04c	TCP	1136	7000 → 49224 [PSH, ACK] Seq=5818 Ack=2318 Wln=64512 Len=1042 Tsv=1-2063913175 TSecr=2799008005 [TCP PDU reassemb
26	0.615547	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:2904:e04c	TCP	1136	7000 → 49224 [PSH, ACK] Seq=6852 Ack=2318 Wln=64512 Len=1042 Tsv=1-2063913175 TSecr=2799008005 [TCP PDU reassemb
27	0.615639	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:2904:e04c	TCP	1136	7000 → 49224 [PSH, ACK] Seq=7894 Ack=2318 Wln=64512 Len=1042 Tsv=1-2063913175 TSecr=2799008005 [TCP PDU reassemb
28	0.615782	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:2904:e04c	TCP	1136	7000 → 49224 [PSH, ACK] Seq=8936 Ack=2318 Wln=64512 Len=1042 Tsv=1-2063913175 TSecr=2799008005 [TCP PDU reassemb
29	0.615943	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:2904:e04c	TCP	1136	7000 → 49224 [ACK] Seq=8978 Ack=2318 Wln=64512 Len=1042 Tsv=1-2063913176 TSecr=2799008005 [TCP PDU reassemb
30	0.616037	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:2904:e04c	TCP	1136	7000 → 49224 [PSH, ACK] Seq=10208 Ack=2318 Wln=64512 Len=1042 Tsv=1-2063913176 TSecr=2799008005 [TCP PDU reassemb
31	0.621006	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:f603:c276	TCP	88	49224 → 7000 [ACK] Seq=2318 Ack=2044 Wln=130608 Len=0 Tsv=1-2799008020 TSecr=2063913175
32	0.618498	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:f603:c276	Gryphon	2044	- Invalid -
33	0.619807	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:f603:c276	TCP	88	49224 → 7000 [ACK] Seq=2318 Ack=7094 Wln=12688 Len=0 Tsv=1-2799008021 TSecr=2063913175
34	0.620642	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:2904:e04c	TCP	4372	7000 → 49224 [PSH, ACK] Seq=14918 Ack=2318 Wln=64512 Len=4288 Tsv=1-2063913180 TSecr=2799008021
35	0.620830	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:2904:e04c	TCP	4372	7000 → 49224 [PSH, ACK] Seq=19282 Ack=2318 Wln=64512 Len=4288 Tsv=1-2063913180 TSecr=2799008021
36	0.620840	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:2904:e04c	TCP	4372	7000 → 49224 [PSH, ACK] Seq=23486 Ack=2318 Wln=64512 Len=4288 Tsv=1-2063913180 TSecr=2799008021
37	0.620870	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:2904:e04c	TCP	1436	7000 → 49224 [PSH, ACK] Seq=27778 Ack=2318 Wln=64512 Len=1342 Tsv=1-2063913180 TSecr=2799008021
38	0.620945	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:f603:c276	TCP	88	[TCP window update] 49224 → 7000 [ACK] Seq=2318 Ack=7094 Wln=131872 Len=0 Tsv=1-2799008021 TSecr=2063913175
39	0.620953	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:f603:c276	TCP	88	49224 → 7000 [ACK] Seq=2318 Ack=12652 Wln=126512 Len=0 Tsv=1-2799008021 TSecr=2063913176
40	0.624031	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:f603:c276	TCP	88	[TCP window update] 49224 → 7000 [ACK] Seq=2318 Ack=12652 Wln=131872 Len=0 Tsv=1-2799008023 TSecr=2063913176
41	0.624042	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:f603:c276	TCP	88	49224 → 7000 [ACK] Seq=2318 Ack=14818 Wln=126256 Len=0 Tsv=1-2799008023 TSecr=2063913176
42	0.625107	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:f603:c276	TCP	88	49224 → 7000 [ACK] Seq=2318 Ack=19202 Wln=126848 Len=0 Tsv=1-2799008027 TSecr=2063913180
43	0.625564	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:f603:c276	TCP	88	[TCP window update] 49224 → 7000 [ACK] Seq=2318 Ack=19202 Wln=131872 Len=0 Tsv=1-2799008027 TSecr=2063913180
44	0.626623	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:f603:c276	TCP	88	49224 → 7000 [ACK] Seq=2318 Ack=27778 Wln=122560 Len=0 Tsv=1-2799008030 TSecr=2063913180
45	0.629991	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:f603:c276	TCP	88	[TCP window update] 49224 → 7000 [ACK] Seq=2318 Ack=27778 Wln=131872 Len=0 Tsv=1-2799008031 TSecr=2063913180
46	0.630041	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:f603:c276	TCP	88	49224 → 7000 [ACK] Seq=2318 Ack=29112 Wln=129792 Len=0 Tsv=1-2799008031 TSecr=2063913180
47	0.630942	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:f603:c276	TCP	216	49224 → 7000 [PSH, ACK] Seq=2318 Ack=29112 Wln=131872 Len=128 Tsv=1-2799008031 TSecr=2063913180
48	0.643552	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:2904:e04c	TCP	156	7000 → 49224 [PSH, ACK] Seq=29112 Ack=2446 Wln=64512 Len=70 Tsv=1-2063913203 TSecr=2799008031
49	0.646454	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:f603:c276	TCP	88	49224 → 7000 [ACK] Seq=2446 Ack=29182 Wln=131808 Len=0 Tsv=1-2799008040 TSecr=2063913203
50	0.670843	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:f603:c276	TCP	612	49224 → 7000 [PSH, ACK] Seq=2446 Ack=29182 Wln=131872 Len=525 Tsv=1-2799008040 TSecr=2063913203
51	0.723850	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:2904:e04c	TCP	88	49224 → 7000 [ACK] Seq=29182 Ack=2971 Wln=64512 Len=239 Tsv=1-2063913204 TSecr=2799008040
52	0.724644	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:2904:e04c	TCP	327	7000 → 49224 [PSH, ACK] Seq=2971 Ack=29421 Wln=131872 Len=553 Tsv=1-2799008130 TSecr=2063913204
53	0.727050	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:f603:c276	TCP	88	49224 → 7000 [ACK] Seq=29421 Ack=3324 Wln=64512 Len=0 Tsv=1-2799008130 TSecr=2799008130
54	0.728793	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:f603:c276	TCP	156	7000 → 49224 [PSH, ACK] Seq=29421 Ack=3324 Wln=64512 Len=70 Tsv=1-2063913209 TSecr=2799008130
55	1.042287	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:2904:e04c	TCP	88	49224 → 7000 [ACK] Seq=3324 Ack=29491 Wln=131808 Len=0 Tsv=1-2799008445 TSecr=2063913209
56	1.052851	Fc80:14d4:eeca:2904:e04c	Fc80:14d4:eeca:f603:c276	TCP	88	49224 → 7000 [ACK] Seq=3324 Ack=29491 Wln=131808 Len=0 Tsv=1-2799008445 TSecr=2063913209

使用 Wireshark 看到，这边仅仅是 TCP 裸流，我们没法知道手机在 wifi 上和车机是怎么交互的，所以我们需要处理。下面说明下处理过程，wifi 过程的 Iap2 和蓝牙不一样，无论是控制还是普通的音频数据都会加密。所以我们需要先进行解密，而解密均是需要密钥的，这样就可以得出第一步就是保存私钥。

Pcap文件 预处理

私钥保存

```
#if (TARGET_OS_ANDROID)
#include <errno.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <sys/system_properties.h>
#include <unistd.h>
#ifndef PROP_VALUE_MAX
#define PROP_VALUE_MAX 92
#endif
#define kPairVerifyKeyDumpProperty "debug.carplay.save_pair_keys"
#define kPairVerifyKeyDumpPath "/data/local/tmp/carplay_pair_verify_keys.txt"
#endif

#if (TARGET_OS_ANDROID)
static boolean _PairVerifyKeyDumpEnabled(void)
```

```

{
    char value[PROP_VALUE_MAX];
    int n;

    n = __system_property_get(kPairVerifyKeyDumpProperty, value);
    if (n <= 0)
        return (false);
    return ((strcmp(value, "true") == 0) || (strcmp(value, "1") == 0));
}

static void _AppendKeyLine(char *buf, size_t cap, size_t *used, const char *name,
const uint8_t key[32])
{
    char hex[65];
    int n;

    if (*used >= cap)
        return;
    DataToHexCString(key, 32, hex);
    n = snprintf(&buf[*used], cap - *used, "%s=%s\n", name, hex);
    MemZeroSecure(hex, sizeof(hex));
    if ((n > 0) && ((size_t)n < (cap - *used)))
        *used += (size_t)n;
}

static void _SavePairVerifyKeysIfEnabled(PairingSessionRef session, const uint8_t
controlRead[32], const uint8_t controlWrite[32])
{
    uint8_t eventsRead[32];
    uint8_t eventsWrite[32];
    char buf[768];
    size_t used = 0;
    int fd;
    int n;
    OSStatus deriveErr;

    if (!_PairVerifyKeyDumpEnabled())
        return;

    MemZeroSecure(eventsRead, sizeof(eventsRead));
    MemZeroSecure(eventsWrite, sizeof(eventsWrite));
    deriveErr = PairingSessionDeriveKey(session, kAirPlayPairingEventsKeySaltPtr,
kAirPlayPairingEventsKeySaltLen,
kAirPlayPairingEventsKeyReadInfoPtr,
kAirPlayPairingEventsKeyReadInfoLen,
sizeof(eventsRead), eventsRead);

    if (!deriveErr)
    {
        deriveErr = PairingSessionDeriveKey(session,
kAirPlayPairingEventsKeySaltPtr, kAirPlayPairingEventsKeySaltLen,
kAirPlayPairingEventsKeyWriteInfoPtr,
kAirPlayPairingEventsKeyWriteInfoLen,
sizeof(eventsWrite), eventsWrite);
    }

    n = snprintf(buf, sizeof(buf),

```

```

        "# phone->car control: control_write\n"
        "# car->phone control: control_read\n"
        "# phone->car events: events_read\n"
        "# car->phone events: events_write\n");
    if (n > 0)
        used = (size_t)n;

    _AppendKeyLine(buf, sizeof(buf), &used, "control_write", controlWrite);
    _AppendKeyLine(buf, sizeof(buf), &used, "control_read", controlRead);
    if (!deriveErr)
    {
        _AppendKeyLine(buf, sizeof(buf), &used, "events_read", eventsRead);
        _AppendKeyLine(buf, sizeof(buf), &used, "events_write", eventsWrite);
    }

    fd = open(kPairVerifyKeyDumpPath, O_CREAT | O_TRUNC | O_WRONLY, 0644);
    if (fd < 0)
    {
        aprs_olog(kLogLevelWarning, "### pair-verify key dump open failed
errno=%d\n", errno);
    }
    else
    {
        if (write(fd, buf, used) != (ssize_t)used)
            aprs_olog(kLogLevelWarning, "### pair-verify key dump write failed
errno=%d\n", errno);
        else
            aprs_olog(kLogLevelNotice, "pair-verify keys saved to %s\n",
kPairVerifyKeyDumpPath);
        (void)fchmod(fd, 0644);
        close(fd);
    }

    MemZeroSecure(buf, sizeof(buf));
    MemZeroSecure(eventsRead, sizeof(eventsRead));
    MemZeroSecure(eventsWrite, sizeof(eventsWrite));
}
#endif

```

之后在 `_HandlePairVerifyHomeKitCompletion(HTTPMessageRef inMsg)` 实现中调用

`_SavePairVerifyKeysIfEnabled` 方法即可。这样编译出 `so` 并且设置

`setprop debug.carplay.save_pair_keys true` 连接无线CarPlay之后就会将私钥保存为：`/data/local/tmp/carplay_pair_verify_keys.txt`

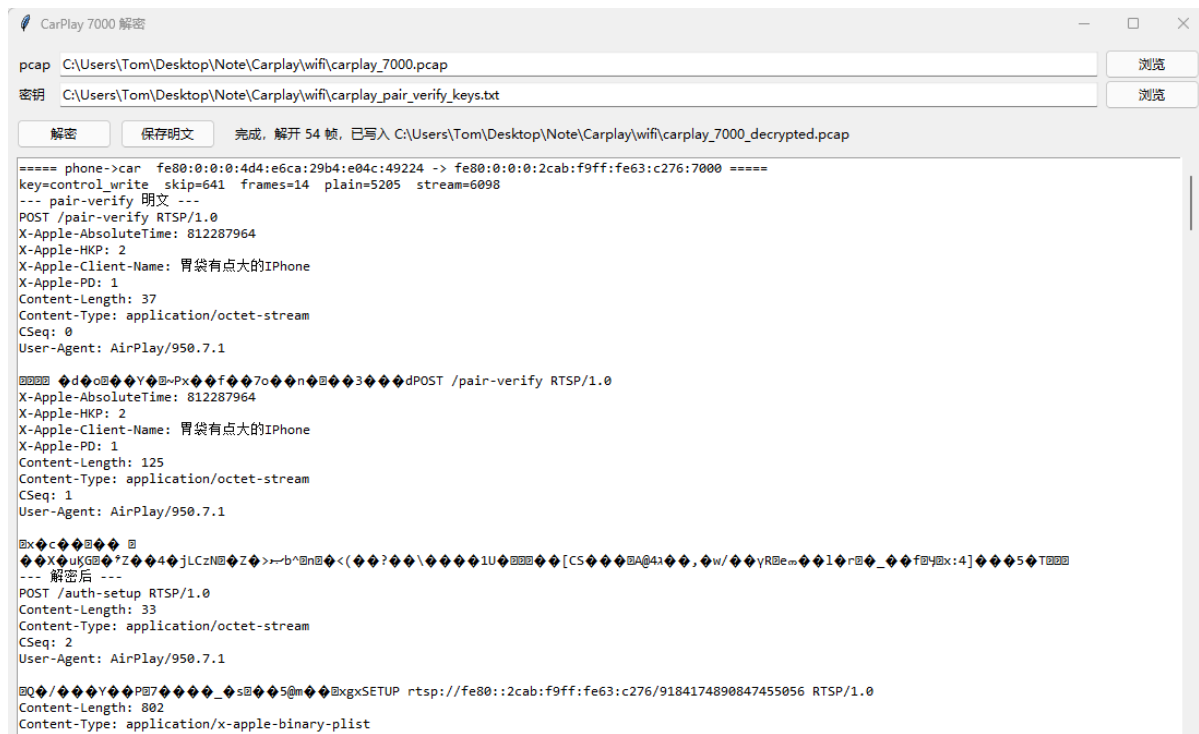
```
# phone->car control: control_write
# car->phone control: control_read
# phone->car events: events_read
# car->phone events: events_write
control_write=807cf03cbff0c8660744b57505557eec564c705090a13e8cf4572cdf3b11790e
control_read=a9cc303a7d1f256f1dad918d42de95b87cbc7ffa7fca2b44228ff94923ce4f2e
events_read=a3be69dd1c818cb2df2a498d14640177e690d7ab4e0983488ed22419d35b9d5f
events_write=017760abe6ff5f251ce15c73f663ef1752918cfbb04f0aa660599edb375db619
```

这里针对于这次连接的私钥信息，保存到 `txt` 中。

Python 解密 pcap

这里定义了编写了 `Python` 代码来做了 `pcap` 的解密。详细代码在：[解码Python](#)

运行解码 `python` 代码之后并且选择 `pcap` 以及私钥文件后



解密完成，解密完成之后的 `pcap` 被写入到 `carplay_7000_decrypted.pcap` 中。后续对于无线 `CarPlay` `wifi` 阶段都将基于解密之后的 `pcap` 来分析。至此，准备工作完成。

No.	Time	Source	Destination	Protocol	Length
1	0.000000	fe80::44a:ec0a:29b4:e04c	fe80::44a:ec0a:29b4:e04c	TCP	100
2	0.000218	fe80::44a:ec0a:29b4:e04c	fe80::44a:ec0a:29b4:e04c	TCP	100
3	0.000479	fe80::44a:ec0a:29b4:e04c	fe80::44a:ec0a:29b4:e04c	TCP	100
4	0.001111	fe80::44a:ec0a:29b4:e04c	fe80::44a:ec0a:29b4:e04c	TCP	100
5	0.009324	fe80::44a:ec0a:29b4:e04c	fe80::44a:ec0a:29b4:e04c	TCP	100
6	0.012237	fe80::44a:ec0a:29b4:e04c	fe80::44a:ec0a:29b4:e04c	TCP	100
7	0.016290	fe80::44a:ec0a:29b4:e04c	fe80::44a:ec0a:29b4:e04c	TCP	100
8	0.025379	fe80::44a:ec0a:29b4:e04c	fe80::44a:ec0a:29b4:e04c	TCP	100
9	0.025447	fe80::44a:ec0a:29b4:e04c	fe80::44a:ec0a:29b4:e04c	TCP	100
10	0.030380	fe80::44a:ec0a:29b4:e04c	fe80::44a:ec0a:29b4:e04c	TCP	100
11	0.032871	fe80::44a:ec0a:29b4:e04c	fe80::44a:ec0a:29b4:e04c	TCP	100
12	0.036120	fe80::44a:ec0a:29b4:e04c	fe80::44a:ec0a:29b4:e04c	TCP	100
13	0.040174	fe80::44a:ec0a:29b4:e04c	fe80::44a:ec0a:29b4:e04c	TCP	100
14	0.046079	fe80::44a:ec0a:29b4:e04c	fe80::44a:ec0a:29b4:e04c	TCP	100
15	0.533874	fe80::44a:ec0a:29b4:e04c	fe80::44a:ec0a:29b4:e04c	TCP	100
16	0.551156	fe80::44a:ec0a:29b4:e04c	fe80::44a:ec0a:29b4:e04c	TCP	100
17	0.551376	fe80::44a:ec0a:29b4:e04c	fe80::44a:ec0a:29b4:e04c	TCP	100
18	0.570400	fe80::44a:ec0a:29b4:e04c	fe80::44a:ec0a:29b4:e04c	TCP	100
19	0.586852	fe80::44a:ec0a:29b4:e04c	fe80::44a:ec0a:29b4:e04c	TCP	100
20	0.602662	fe80::44a:ec0a:29b4:e04c	fe80::44a:ec0a:29b4:e04c	TCP	100
21	0.614993	fe80::44a:ec0a:29b4:e04c	fe80::44a:ec0a:29b4:e04c	TCP	100
22	0.615173	fe80::44a:ec0a:29b4:e04c	fe80::44a:ec0a:29b4:e04c	TCP	100
23	0.615272	fe80::44a:ec0a:29b4:e04c	fe80::44a:ec0a:29b4:e04c	TCP	100
24	0.615365	fe80::44a:ec0a:29b4:e04c	fe80::44a:ec0a:29b4:e04c	TCP	100
25	0.615457	fe80::44a:ec0a:29b4:e04c	fe80::44a:ec0a:29b4:e04c	TCP	100
26	0.615547	fe80::44a:ec0a:29b4:e04c	fe80::44a:ec0a:29b4:e04c	TCP	100
27	0.615639	fe80::44a:ec0a:29b4:e04c	fe80::44a:ec0a:29b4:e04c	TCP	100
28	0.615782	fe80::44a:ec0a:29b4:e04c	fe80::44a:ec0a:29b4:e04c	TCP	100
29	0.615943	fe80::44a:ec0a:29b4:e04c	fe80::44a:ec0a:29b4:e04c	TCP	100
30	0.616037	fe80::44a:ec0a:29b4:e04c	fe80::44a:ec0a:29b4:e04c	TCP	100
31	0.616106	fe80::44a:ec0a:29b4:e04c	fe80::44a:ec0a:29b4:e04c	TCP	100
32	0.618498	fe80::44a:ec0a:29b4:e04c	fe80::44a:ec0a:29b4:e04c	TCP	100
33	0.619087	fe80::44a:ec0a:29b4:e04c	fe80::44a:ec0a:29b4:e04c	TCP	100
34	0.620422	fe80::44a:ec0a:29b4:e04c	fe80::44a:ec0a:29b4:e04c	TCP	100
35	0.620390	fe80::44a:ec0a:29b4:e04c	fe80::44a:ec0a:29b4:e04c	TCP	100
36	0.620408	fe80::44a:ec0a:29b4:e04c	fe80::44a:ec0a:29b4:e04c	TCP	100
37	0.620970	fe80::44a:ec0a:29b4:e04c	fe80::44a:ec0a:29b4:e04c	TCP	100
38	0.620645	fe80::44a:ec0a:29b4:e04c	fe80::44a:ec0a:29b4:e04c	TCP	100
39	0.613835	fe80::44a:ec0a:29b4:e04c	fe80::44a:ec0a:29b4:e04c	TCP	100

上述是解密之前

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	fe80::4d4a:e6ca:29b4:e04c	fe80::12cab:f9ff:fe63:c276	TCP	76	49224 → 7000 [SYN] Seq=0 Win=65535 Len=0
2	0.001000	fe80::12cab:f9ff:fe63:c276	fe80::4d4a:e6ca:29b4:e04c	TCP	76	7000 → 49224 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0
3	0.002000	fe80::4d4a:e6ca:29b4:e04c	fe80::12cab:f9ff:fe63:c276	TCP	76	49224 → 7000 [ACK] Seq=1 Ack=1 Win=65535 Len=0
4	0.003000	fe80::4d4a:e6ca:29b4:e04c	fe80::12cab:f9ff:fe63:c276	RTSP	352	Continuation
5	0.004000	fe80::12cab:f9ff:fe63:c276	fe80::4d4a:e6ca:29b4:e04c	RTSP	348	Reply: RTSP/1.0 200 OK
6	0.005000	fe80::4d4a:e6ca:29b4:e04c	fe80::12cab:f9ff:fe63:c276	RTSP	441	Continuation
7	0.006000	fe80::12cab:f9ff:fe63:c276	fe80::4d4a:e6ca:29b4:e04c	RTSP	190	Reply: RTSP/1.0 200 OK
8	0.007000	fe80::4d4a:e6ca:29b4:e04c	fe80::12cab:f9ff:fe63:c276	RTSP	236	Continuation
9	0.008000	fe80::12cab:f9ff:fe63:c276	fe80::4d4a:e6ca:29b4:e04c	RTSP	901	Reply: RTSP/1.0 200 OK
10	0.009000	fe80::4d4a:e6ca:29b4:e04c	fe80::12cab:f9ff:fe63:c276	RTSP	1056	SETUP rtsp://fe80::12cab:f9ff:fe63:c276/9184174890847455056 RTSP/1.0
11	0.010000	fe80::12cab:f9ff:fe63:c276	fe80::4d4a:e6ca:29b4:e04c	RTSP	470	Reply: RTSP/1.0 200 OK
12	0.011000	fe80::4d4a:e6ca:29b4:e04c	fe80::12cab:f9ff:fe63:c276	RTSP	558	Continuation
13	0.012000	fe80::12cab:f9ff:fe63:c276	fe80::4d4a:e6ca:29b4:e04c	RTSP	27060	Reply: RTSP/1.0 200 OK
14	0.013000	fe80::4d4a:e6ca:29b4:e04c	fe80::12cab:f9ff:fe63:c276	RTSP	186	RECORD rtsp://fe80::12cab:f9ff:fe63:c276/9184174890847455056 RTSP/1.0
15	0.014000	fe80::12cab:f9ff:fe63:c276	fe80::4d4a:e6ca:29b4:e04c	RTSP	128	Reply: RTSP/1.0 200 OK
16	0.015000	fe80::4d4a:e6ca:29b4:e04c	fe80::12cab:f9ff:fe63:c276	RTSP	583	SETUP rtsp://fe80::12cab:f9ff:fe63:c276/9184174890847455056 RTSP/1.0
17	0.016000	fe80::12cab:f9ff:fe63:c276	fe80::4d4a:e6ca:29b4:e04c	RTSP	297	Reply: RTSP/1.0 200 OK
18	0.017000	fe80::4d4a:e6ca:29b4:e04c	fe80::12cab:f9ff:fe63:c276	RTSP	411	SET_PARAMETER rtsp://fe80::12cab:f9ff:fe63:c276/9184174890847455056 RTSP/1.0
19	0.018000	fe80::12cab:f9ff:fe63:c276	fe80::4d4a:e6ca:29b4:e04c	RTSP	128	Reply: RTSP/1.0 200 OK
20	0.019000	fe80::4d4a:e6ca:29b4:e04c	fe80::12cab:f9ff:fe63:c276	RTSP	496	Continuation
21	0.020000	fe80::12cab:f9ff:fe63:c276	fe80::4d4a:e6ca:29b4:e04c	RTSP	238	Reply: RTSP/1.0 200 OK
22	0.021000	fe80::4d4a:e6ca:29b4:e04c	fe80::12cab:f9ff:fe63:c276	RTSP	351	Continuation
23	0.022000	fe80::12cab:f9ff:fe63:c276	fe80::4d4a:e6ca:29b4:e04c	RTSP	143	Reply: RTSP/1.0 0
24	0.023000	fe80::4d4a:e6ca:29b4:e04c	fe80::12cab:f9ff:fe63:c276	RTSP	325	Continuation
25	0.024000	fe80::12cab:f9ff:fe63:c276	fe80::4d4a:e6ca:29b4:e04c	RTSP	239	Reply: RTSP/1.0 200 OK
26	0.025000	fe80::4d4a:e6ca:29b4:e04c	fe80::12cab:f9ff:fe63:c276	RTSP	497	Continuation
27	0.026000	fe80::12cab:f9ff:fe63:c276	fe80::4d4a:e6ca:29b4:e04c	RTSP	239	Reply: RTSP/1.0 200 OK
28	0.027000	fe80::4d4a:e6ca:29b4:e04c	fe80::12cab:f9ff:fe63:c276	RTSP	584	SETUP rtsp://fe80::12cab:f9ff:fe63:c276/9184174890847455056 RTSP/1.0
29	0.028000	fe80::12cab:f9ff:fe63:c276	fe80::4d4a:e6ca:29b4:e04c	RTSP	316	Reply: RTSP/1.0 200 OK
30	0.029000	fe80::4d4a:e6ca:29b4:e04c	fe80::12cab:f9ff:fe63:c276	RTSP	422	SETUP rtsp://fe80::12cab:f9ff:fe63:c276/9184174890847455056 RTSP/1.0
31	0.030000	fe80::12cab:f9ff:fe63:c276	fe80::4d4a:e6ca:29b4:e04c	RTSP	301	Reply: RTSP/1.0 200 OK
32	0.031000	fe80::4d4a:e6ca:29b4:e04c	fe80::12cab:f9ff:fe63:c276	RTSP	422	SETUP rtsp://fe80::12cab:f9ff:fe63:c276/9184174890847455056 RTSP/1.0
33	0.032000	fe80::12cab:f9ff:fe63:c276	fe80::4d4a:e6ca:29b4:e04c	RTSP	282	Reply: RTSP/1.0 200 OK
34	0.033000	fe80::4d4a:e6ca:29b4:e04c	fe80::12cab:f9ff:fe63:c276	RTSP	142	Continuation
35	0.034000	fe80::12cab:f9ff:fe63:c276	fe80::4d4a:e6ca:29b4:e04c	RTSP	566	Reply: RTSP/1.0 200 OK
36	0.035000	fe80::4d4a:e6ca:29b4:e04c	fe80::12cab:f9ff:fe63:c276	TCP	76	49224 → 7000 [FIN, ACK] Seq=5847 Ack=30631 Win=65535 Len=0
37	0.036000	fe80::12cab:f9ff:fe63:c276	fe80::4d4a:e6ca:29b4:e04c	TCP	76	7000 → 49224 [FIN, ACK] Seq=30631 Ack=5848 Win=65535 Len=0

上述是解密之后，对比可以看到，我们可以看到更多 RTSP 的协议语义、包括具体的 SETUP、RECORD 协议等等。

疑惑备注

这里其实在解析时存在一些疑惑。

1. 为什么这里 RTSP 还是会存在 Continuation，这个实际上是 Wireshark 的解析器行为，默认上，Wireshark 解析时只支持 RTSP 的蓝皮书协议字段，但 AirPlay 是对 RTSP 协议栈做了一些扩展。

```
static HTTPStatus _requestProcessOptions(AirPlayReceiverConnectionRef inCnx)
{
    HTTPMessageRef response = inCnx->httpCnx->responseMsg;
    HTTPHeader_SetField(&response->header, kHTTPHeader_Public, "ANNOUNCE,
SETUP, RECORD, PAUSE, FLUSH, TEARDOWN, OPTIONS, POST, GET, PUT,
GET_PARAMETER");
    return (kHTTPStatus_OK);
}
```

这里可以清楚的看到，这里的 POST / GET / PUT 都是 RTSP 的扩展，而非官方协议支持的，所以 Wireshark 解析为 Continuation 属于正常行为。

2. 为什么在 AirPlay 协议栈中，使用的是 HTTP 报文格式进行解析的。CarPlay 控制到底是基于 HTTP 还是 RTSP？

上述 Pcap 中可以解答这一点，CarPlay 会话控制均是基于 RTSP 协议来实现，至于和 HTTP 协议有一点相似，是因为 RTSP 从诞生起就是按照 HTTP/1.1 的格式定的，所以 HTTP 的报文解析器可以完全解析 RTSP 的报文。而且第一点中，实现的 POST / PUT / GET 很容易和 HTTP 协议混在一起。但是实际上，CarPlay 控制完全就是基于 RTSP。

无线CarPlay Wifi阶段连接流程

wifi 连接流程中：整体流程为：tcp 三次握手 -> HttpServer 接收 pair-verify -> HttpServer 第二次的接收到 pair-verify -> HttpServer 接收到 auth-setup

-> Httpserver 接收到 GET /info -> 之后在 Httpserver 会接收到很多从 Controller 发过来 POST 请求。

carplay_7000_decrypted.pcap 可详细的展示这块细节

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	fe80::4d4:edca:29b4:e04c	fe80::2cab:f9ff:fe63:c276	TCP	76	49224 → 7000 [SYN] Seq=0 Win=65535 Len=0
2	0.001000	fe80::2cab:f9ff:fe63:c276	fe80::4d4:edca:29b4:e04c	TCP	76	7000 → 49224 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0
3	0.002000	fe80::4d4:edca:29b4:e04c	fe80::2cab:f9ff:fe63:c276	TCP	76	49224 → 7000 [ACK] Seq=1 Ack=1 Win=65535 Len=0
4	0.003000	fe80::4d4:edca:29b4:e04c	fe80::2cab:f9ff:fe63:c276	RTSP	352	Continuation
5	0.004000	fe80::4d4:edca:29b4:e04c	fe80::2cab:f9ff:fe63:c276	RTSP	348	Reply: RTSP/1.0 200 OK
6	0.005000	fe80::4d4:edca:29b4:e04c	fe80::2cab:f9ff:fe63:c276	RTSP	441	Continuation
7	0.006000	fe80::2cab:f9ff:fe63:c276	fe80::4d4:edca:29b4:e04c	RTSP	190	Reply: RTSP/1.0 200 OK
8	0.007000	fe80::4d4:edca:29b4:e04c	fe80::2cab:f9ff:fe63:c276	RTSP	236	Continuation
9	0.008000	fe80::2cab:f9ff:fe63:c276	fe80::4d4:edca:29b4:e04c	RTSP	901	Reply: RTSP/1.0 200 OK
10	0.009000	fe80::4d4:edca:29b4:e04c	fe80::2cab:f9ff:fe63:c276	RTSP	1056	SETUP rtsp://fe80::2cab:f9ff:fe63:c276/9184174890847455056 RTSP/1.0
11	0.010000	fe80::2cab:f9ff:fe63:c276	fe80::4d4:edca:29b4:e04c	RTSP	470	Reply: RTSP/1.0 200 OK
12	0.011000	fe80::4d4:edca:29b4:e04c	fe80::2cab:f9ff:fe63:c276	RTSP	558	Continuation
13	0.012000	fe80::2cab:f9ff:fe63:c276	fe80::4d4:edca:29b4:e04c	RTSP	27608	Reply: RTSP/1.0 200 OK
14	0.013000	fe80::4d4:edca:29b4:e04c	fe80::2cab:f9ff:fe63:c276	RTSP	186	RECORD rtsp://fe80::2cab:f9ff:fe63:c276/9184174890847455056 RTSP/1.0
15	0.014000	fe80::2cab:f9ff:fe63:c276	fe80::4d4:edca:29b4:e04c	RTSP	128	Reply: RTSP/1.0 200 OK
16	0.015000	fe80::4d4:edca:29b4:e04c	fe80::2cab:f9ff:fe63:c276	RTSP	583	SETUP rtsp://fe80::2cab:f9ff:fe63:c276/9184174890847455056 RTSP/1.0
17	0.016000	fe80::2cab:f9ff:fe63:c276	fe80::4d4:edca:29b4:e04c	RTSP	297	Reply: RTSP/1.0 200 OK
18	0.017000	fe80::4d4:edca:29b4:e04c	fe80::2cab:f9ff:fe63:c276	RTSP	411	SET_PARAMETER rtsp://fe80::2cab:f9ff:fe63:c276/9184174890847455056 RTSP/1.0
19	0.018000	fe80::2cab:f9ff:fe63:c276	fe80::4d4:edca:29b4:e04c	RTSP	128	Reply: RTSP/1.0 200 OK
20	0.019000	fe80::4d4:edca:29b4:e04c	fe80::2cab:f9ff:fe63:c276	RTSP	496	Continuation
21	0.020000	fe80::2cab:f9ff:fe63:c276	fe80::4d4:edca:29b4:e04c	RTSP	238	Reply: RTSP/1.0 200 OK
22	0.021000	fe80::4d4:edca:29b4:e04c	fe80::2cab:f9ff:fe63:c276	RTSP	351	Continuation
23	0.022000	fe80::2cab:f9ff:fe63:c276	fe80::4d4:edca:29b4:e04c	RTSP	143	Reply: RTSP/1.0 0
24	0.023000	fe80::4d4:edca:29b4:e04c	fe80::2cab:f9ff:fe63:c276	RTSP	325	Continuation
25	0.024000	fe80::2cab:f9ff:fe63:c276	fe80::4d4:edca:29b4:e04c	RTSP	239	Reply: RTSP/1.0 200 OK
26	0.025000	fe80::4d4:edca:29b4:e04c	fe80::2cab:f9ff:fe63:c276	RTSP	497	Continuation
27	0.026000	fe80::2cab:f9ff:fe63:c276	fe80::4d4:edca:29b4:e04c	RTSP	239	Reply: RTSP/1.0 200 OK
28	0.027000	fe80::4d4:edca:29b4:e04c	fe80::2cab:f9ff:fe63:c276	RTSP	584	SETUP rtsp://fe80::2cab:f9ff:fe63:c276/9184174890847455056 RTSP/1.0
29	0.028000	fe80::2cab:f9ff:fe63:c276	fe80::4d4:edca:29b4:e04c	RTSP	316	Reply: RTSP/1.0 200 OK
30	0.029000	fe80::4d4:edca:29b4:e04c	fe80::2cab:f9ff:fe63:c276	RTSP	422	SETUP rtsp://fe80::2cab:f9ff:fe63:c276/9184174890847455056 RTSP/1.0
31	0.030000	fe80::2cab:f9ff:fe63:c276	fe80::4d4:edca:29b4:e04c	RTSP	381	Reply: RTSP/1.0 200 OK
32	0.031000	fe80::4d4:edca:29b4:e04c	fe80::2cab:f9ff:fe63:c276	RTSP	422	SETUP rtsp://fe80::2cab:f9ff:fe63:c276/9184174890847455056 RTSP/1.0
33	0.032000	fe80::2cab:f9ff:fe63:c276	fe80::4d4:edca:29b4:e04c	RTSP	282	Reply: RTSP/1.0 200 OK
34	0.033000	fe80::4d4:edca:29b4:e04c	fe80::2cab:f9ff:fe63:c276	RTSP	142	Continuation
35	0.034000	fe80::2cab:f9ff:fe63:c276	fe80::4d4:edca:29b4:e04c	RTSP	566	Reply: RTSP/1.0 200 OK
36	0.035000	fe80::4d4:edca:29b4:e04c	fe80::2cab:f9ff:fe63:c276	TCP	76	49224 → 7000 [FIN, ACK] Seq=5847 Ack=38631 Win=65535 Len=0
37	0.036000	fe80::2cab:f9ff:fe63:c276	fe80::4d4:edca:29b4:e04c	TCP	76	7000 → 49224 [FIN, ACK] Seq=38631 Ack=5848 Win=65535 Len=0

这里可以给出 CarPlay wifi 阶段连接流程。

- POST /pair-verify RTSP/1.0
- POST /pair-verify RTSP/1.0
- POST /auth-setup RTSP/1.0
- GET /info RTSP/1.0
- POST /command RTSP/1.0
- 其他标准 RTSP 协议

可以看到 AirPlay 协议栈在 wifi 阶段接收到两次自定义 RTSP POST 信令，并且传递的值均为 pair-verify。这两次 POST 很重要。他是CarPlay安全传输数据的基础的。接收到 pair-verify，AirPlay 协议栈 又接收到 auth-setup，这一次 POST 同样很重要，它可以确保，连接的对端是一个可用 CarPlay设备。

Pair-verify1

Httpserver 会接收到两次 Pair-verify 请求，为了区分这两次 Pair-verify 的 POST 请求，所以这里将第一次的 Pair-verify 称为 Pair-verify1。从整个通信的过程来看，Pair-verify 的过程主要为后面链路做铺垫。此过程即会去和手机协商公钥以及后续传输的签名，后续手机和 Httpserver 的传输数据均为加密的。

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	fe80::4d4:edca:29b4:e04c	fe80::2cab:f9ff:fe63:c276	TCP	76	49224 → 7000 [SYN] Seq=0 Win=65535 Len=0
2	0.001000	fe80::2cab:f9ff:fe63:c276	fe80::4d4:edca:29b4:e04c	TCP	76	7000 → 49224 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0
3	0.002000	fe80::4d4:edca:29b4:e04c	fe80::2cab:f9ff:fe63:c276	TCP	76	49224 → 7000 [ACK] Seq=1 Ack=1 Win=65535 Len=0
4	0.003000	fe80::4d4:edca:29b4:e04c	fe80::2cab:f9ff:fe63:c276	RTSP	352	Continuation
5	0.004000	fe80::2cab:f9ff:fe63:c276	fe80::4d4:edca:29b4:e04c	RTSP	348	Reply: RTSP/1.0 200 OK
6	0.005000	fe80::4d4:edca:29b4:e04c	fe80::2cab:f9ff:fe63:c276	RTSP	441	Continuation
7	0.006000	fe80::2cab:f9ff:fe63:c276	fe80::4d4:edca:29b4:e04c	RTSP	190	Reply: RTSP/1.0 200 OK
8	0.007000	fe80::4d4:edca:29b4:e04c	fe80::2cab:f9ff:fe63:c276	RTSP	236	Continuation
9	0.008000	fe80::2cab:f9ff:fe63:c276	fe80::4d4:edca:29b4:e04c	RTSP	901	Reply: RTSP/1.0 200 OK
10	0.009000	fe80::4d4:edca:29b4:e04c	fe80::2cab:f9ff:fe63:c276	RTSP	1056	SETUP rtsp://fe80::2cab:f9ff:fe63:c276/9184174890847455056 RTSP/1.0
11	0.010000	fe80::2cab:f9ff:fe63:c276	fe80::4d4:edca:29b4:e04c	RTSP	470	Reply: RTSP/1.0 200 OK
12	0.011000	fe80::4d4:edca:29b4:e04c	fe80::2cab:f9ff:fe63:c276	RTSP	558	Continuation
13	0.012000	fe80::2cab:f9ff:fe63:c276	fe80::4d4:edca:29b4:e04c	RTSP	27608	Reply: RTSP/1.0 200 OK
14	0.013000	fe80::4d4:edca:29b4:e04c	fe80::2cab:f9ff:fe63:c276	RTSP	186	RECORD rtsp://fe80::2cab:f9ff:fe63:c276/9184174890847455056 RTSP/1.0
15	0.014000	fe80::2cab:f9ff:fe63:c276	fe80::4d4:edca:29b4:e04c	RTSP	128	Reply: RTSP/1.0 200 OK
16	0.015000	fe80::4d4:edca:29b4:e04c	fe80::2cab:f9ff:fe63:c276	RTSP	583	SETUP rtsp://fe80::2cab:f9ff:fe63:c276/9184174890847455056 RTSP/1.0
17	0.016000	fe80::2cab:f9ff:fe63:c276	fe80::4d4:edca:29b4:e04c	RTSP	297	Reply: RTSP/1.0 200 OK
18	0.017000	fe80::4d4:edca:29b4:e04c	fe80::2cab:f9ff:fe63:c276	RTSP	411	SET_PARAMETER rtsp://fe80::2cab:f9ff:fe63:c276/9184174890847455056 RTSP/1.0
Frame 4: Packet, 352 bytes on wire (2816 bits), 352 bytes captured (2816 bits)						
Linux cooked capture v1						
Internet Protocol Version 6, Src: fe80::4d4:edca:29b4:e04c, Dst: fe80::2cab:f9ff:fe63:c276						
Transmission Control Protocol, Src Port: 49224, Dst Port: 7000, Seq: 1, Ack: 1, Len: 276						
Pair-verify (Continuation)						
POST /pair-verify RTSP/1.0\r\n						
X-Apple-AbsoluteTime: 812287964\r\n						
X-Apple-Model: 2,1\r\n						
X-Apple-Client-Name: 屏幕有点大的iPhone\r\n						
X-Apple-POS: 1\r\n						
Content-Length: 37						
Content-type: application/octet-stream						
CSeq: 0						
User-Agent: AirPlay/950.2.1\r\n						
Data (37 bytes)						
Data: 60810320a6d4ce611d4c5590d157e5070b7a966f8d276fa2886e412879433b0ba964						
[Length: 37]						

从 pcap 中可以看到明显的交互流程，如果需要了解这次传输什么数据就需要去从代码的角度去分析下：

```

status = _requestProcessPairVerify(cnx, inRequest);
|
| - - - - -> PairingSessionCreate() 这个类型为
(kPairingSessionType_VerifyServer) / PairingSessionExchange()
| - - - - -> _VerifyPairingServerExchange()

```

`_VerifyPairingServerExchange` 中涉及到很多密码学的知识，`verify` 这个过程使用状态机来维护这个过程。

M1 状态

```

if( me->state == kPairingStateInvalid ) me->state = kPairVerifyStateM1; //初始状态
机为 m1

err = RandomBytes( me->ourCurveSK, sizeof( me->ourCurveSK ) );

require_noerr( err, exit );

HKDF_SHA512( me->ourCurveSK, sizeof( me->ourCurveSK ),
             kPairVerifyECDHSaltPtr, kPairVerifyECDHSaltLen,
             kPairVerifyECDHInfoPtr, kPairVerifyECDHInfoLen,
             sizeof( me->ourCurveSK ), me->ourCurveSK );
//生成了车机端的私钥，并且将私钥设置保存在 ourCurveSK中。

curve25519( me->ourCurvePK, me->ourCurveSK, NULL );
//上述的过程通过私钥派生成了车机端的公钥，并且将公钥设置保存在 ourCurvePK中，后续会将公钥设置给
Controller。

//这一步骤完成之后，车机的私钥和公钥都已经创建完成。

err = TLV8GetBytes( inputPtr, inputEnd, kTLVType_PublicKey, 32, 32, me-
>peerCurvePK, NULL, NULL );
//从POST 中拿到携带的手机端的公钥。
require_noerr( err, exit );

curve25519( me->sharedSecret, me->ourCurveSK, me->peerCurvePK );
//车机的私钥和手机的公钥生成了共享密码。后续数据传输会使用这个共享密码来加密数据。

```

所以上面图中 `Pair-verify` 携带参数是手机侧携带的公钥。

至此，`Verify M1` 状态结束，之后进入到 `M2` 状态。总结下来，`M1` 状态主要是接收手机传递过来公钥的，后续使用 `curve25519` 算法来生成公钥、私钥、以及共享密钥。

M2 状态

M1状态中生成了共享密钥，加密解密都是均使用共享密钥去完成。但是到这一步仅仅共享密钥是不够安全的，会受到中间人攻击。所以会存在M2状态。M2状态中引入了签名，以便来进一步加强安全性来确保通信就是对方。

```

// M2: Accessory -> Controller -- Start Response.

// Generate signature of our info.

```

```

ForgetPtrLen( &me->activeIdentifierPtr, &me->activeIdentifierLen );

//通过私钥和公钥生成签名标识。
err = PairingSessionCopyIdentity( me, false, &me->activeIdentifierPtr, me-
>ourEdPK, me->ourEdSK );
require_noerr_quiet( err, exit );
me->activeIdentifierLen = strlen( me->activeIdentifierPtr );
require_action( me->activeIdentifierLen > 0, exit, err = kIDErr );

len = 32 + me->activeIdentifierLen + 32;
storage = (uint8_t *) malloc( len );
require_action( storage, exit, err = kNoMemoryErr );
memcpy( &storage[ 0 ], me->ourCurvePK, 32 );
memcpy( &storage[ 32 ], me->activeIdentifierPtr, me->activeIdentifierLen );
memcpy( &storage[ 32 + me->activeIdentifierLen ], me->peerCurvePK, 32 );

//使用Ed25519来生成签名。
Ed25519_sign( sig, storage, len, me->ourEdPK, me->ourEdSK );
ForgetMem( &storage );

// Build sub-TLV of accessory's info and encrypt it.

err = TLV8BufferAppend( &etlv, kTLVType_Identifier, me->activeIdentifierPtr, me-
>activeIdentifierLen );
require_noerr( err, exit );
err = TLV8BufferAppend( &etlv, kTLVType_Signature, sig, 64 );
require_noerr( err, exit );

storage = (uint8_t *) malloc( etlv.len + 16 );
require_action( storage, exit, err = kNoMemoryErr );
HKDF_SHA512( me->sharedSecret, sizeof( me->sharedSecret ),
    kPairVerifyEncryptSaltPtr, kPairVerifyEncryptSaltLen,
    kPairVerifyEncryptInfoPtr, kPairVerifyEncryptInfoLen,
    sizeof( me->key ), me->key );

//sharedSecret使用共享密钥来加密签名。
chacha20_poly1305_encrypt_all_64x64( me->key, (const uint8_t *) "PV-Msg02", NULL,
0,
    etlv.ptr, etlv.len, storage, &storage[ etlv.len ] );
err = TLV8BufferAppend( &tlv, kTLVType_EncryptedData, storage, etlv.len + 16 );
require_noerr( err, exit );
ForgetMem( &storage );

me->state = kPairVerifyStateM2;

err = TLV8BufferAppend( &tlv, kTLVType_State, &me->state, sizeof( me->state ) );
require_noerr( err, exit );
err = TLV8BufferAppend( &tlv, kTLVType_PublicKey, me->ourCurvePK, 32 );
require_noerr( err, exit );
err = TLV8BufferDetach( &tlv, outOutputPtr, outOutputLen );
require_noerr( err, exit );

//最终将签名加上车机上公钥一起通过Http返回给到手机。
pair_uLog( me, kLogLevelTrace, "Pair-verify server M2 -- start response\n%?
{end}%1{tlv8}\n",

```

```

        !log_category_enabled( me->ucat, kLogLevelVerbose ), kTLVDescriptors,
        *outOutputPtr, *outOutputLen );
me->state = kPairVerifyStateM3;
break;

```

至此，连接的 Pair-verify1 阶段结束，从整个过程来看，主要是 M1 阶段是接收到手机传递过来的公钥、生成自己公钥，以及自己生成共享密钥。这些准备工作完成之后，进入到 M2 阶段，这个阶段会使用 Ed25519 算法来会生成设备的签名，后使用 chacha20_poly1305_encrypt_all_64x64 算法来对签名来加密，最终使用加密后的签名回复给手机。

The screenshot shows a Wireshark packet capture of an RTSP session. The selected packet is a 'Response: RTSP/1.0 200 OK' from the server to the client. The packet details pane shows the 'Data' field containing a large block of encrypted data (159 bytes). The packet bytes pane shows the raw hexadecimal data of the encrypted payload.

上述传递 159 字节就是经过公钥、签名数据。

Pair-verify 2

The screenshot shows a Wireshark packet capture of an RTSP session. The selected packet is a 'POST /pair-verify RTSP/1.0' request from the client to the server. The packet details pane shows the 'Data' field containing a large block of encrypted data (159 bytes). The packet bytes pane shows the raw hexadecimal data of the encrypted payload.

verify 2 RTSP Post 请求即对应着后续两个状态，即 M3、M4。M3 主要是处理从手机过来签名数据，确定是否是通信目标是合法的以及共享密钥是否正确等等。

M3 状态

```

case kPairVerifyStateM3:
    pair_ulog( me, kLogLevelTrace, "Pair-verify server M3 -- finish request\n"?
    {end}%1{tlv8}\n",
    !log_category_enabled( me->ucat, kLogLevelVerbose ), kTLVDescriptors,
    inInputPtr, (int) inInputLen );

    // verify and decrypt sub-TLV.

```

//这里从HttpBody中拿到加密之后的二进制数据。

```

    eptr = TLV8CopyCoalesced( inputPtr, inputEnd, kTLVType_EncryptedData, &elen,
    NULL, &err );
    require_noerr( err, exit );
    require_action( elen > 16, exit, err = kSizeErr );
    elen -= 16;
    eend = eptr + elen;

    //使用共享密钥进行解密
    err = chacha20_poly1305_decrypt_all_64x64( me->key, (const uint8_t *) "pv-
Msg03", NULL, 0, eptr, elen, eptr, eend );
    if( err )
    {
        pair_olog( me, kLogLevelWarning, "### Pair-verify server bad auth tag\n"
    );

        err = TLV8BufferAppendUInt64( &tlv, kTLVType_Error,
    kTLVError_Authentication );
        require_noerr( err, exit );
        err = TLV8BufferAppendUInt64( &tlv, kTLVType_State, kPairVerifyStateM4 );
        require_noerr( err, exit );
        err = TLV8BufferDetach( &tlv, outOutputPtr, outOutputLen );
        require_noerr( err, exit );

        _PairingSessionReset( me );
        goto exit;
    }

    //拿到手机端给过来的签名问题
    // Look up accessory's LTPK.

    ForgetPtrLen( &me->peerIdentifierPtr, &me->peerIdentifierLen );
    me->peerIdentifierPtr = (char*)TLV8CopyCoalesced( eptr, eend,
    kTLVType_Identifier, &me->peerIdentifierLen, NULL, &err );
    require_noerr( err, exit );
    require_action( me->peerIdentifierLen > 0, exit, err = kSizeErr );

    err = PairingSessionFindPeer( me, me->peerIdentifierPtr, me-
    >peerIdentifierLen, me->peerEdPK );
    if( err )
    {
        pair_olog( me, kLogLevelWarning, "### Pair-verify server unknown peer:
    %.*s\n",
            (int) me->peerIdentifierLen, me->peerIdentifierPtr );

        err = TLV8BufferAppendUInt64( &tlv, kTLVType_Error,
    kTLVError_Authentication );
        require_noerr( err, exit );
        err = TLV8BufferAppendUInt64( &tlv, kTLVType_State, kPairVerifyStateM4 );
        require_noerr( err, exit );
        err = TLV8BufferDetach( &tlv, outOutputPtr, outOutputLen );
        require_noerr( err, exit );

        _PairingSessionReset( me );
        goto exit;
    }
    ForgetMem( &storage );

```

```

// Verify signature of controller's info.
//验证相对的签名。

err = TLV8GetBytes( eptr, eend, kTLVType_Signature, 64, 64, sig, NULL, NULL
);
require_noerr( err, exit );

len = 32 + me->peerIdentifierLen + 32;
storage = (uint8_t *) malloc( len );
require_action( storage, exit, err = kNoMemoryErr );
memcpy( &storage[ 0 ], me->peerCurvePK, 32 );
memcpy( &storage[ 32 ], me->peerIdentifierPtr, me->peerIdentifierLen );
memcpy( &storage[ 32 + me->peerIdentifierLen ], me->ourCurvePK, 32 );
err = Ed25519_verify( storage, len, sig, me->peerEdPK );
if( err )
{
    pair_olog( me, kLogLevelWarning, "### Pair-verify server bad signature:
    %m\n", err );

    err = TLV8BufferAppendUInt64( &tlv, kTLVType_Error,
    kTLVError_Authentication );
    require_noerr( err, exit );
    err = TLV8BufferAppendUInt64( &tlv, kTLVType_State, kPairVerifyStateM4 );
    require_noerr( err, exit );
    err = TLV8BufferDetach( &tlv, outOutputPtr, outOutputLen );
    require_noerr( err, exit );

    _PairingSessionReset( me );
    goto exit;
}
ForgetMem( &storage );

me->state = kPairVerifyStateM4;

```

从源码中可以清晰的看到，M3 这个过程主要时对手机端进行签名认证。之后状态机流转到 M4。

M4 状态

```

// M4: Accessory -> Controller -- Finish Response.

//将state作为 pair-verify 2的返回值。
err = TLV8BufferAppend( &tlv, kTLVType_State, &me->state, sizeof( me->state ) );
require_noerr( err, exit );
err = TLV8BufferDetach( &tlv, outOutputPtr, outOutputLen );
require_noerr( err, exit );

pair_olog( me, kLogLevelTrace, "Pair-verify server M4 -- finish response\n%?
{end}%1{tlv8}\n",
    !log_category_enabled( me->ucat, kLogLevelVerbose ), kTLVDescriptors,
    *outOutputPtr, *outOutputLen );
me->state = kPairVerifyStateDone;
done = true;
pair_olog( me, kLogLevelTrace, "Pair-verify server done\n" );

```

```
break;
```

Pair-verify 流程总结

Pair-verify 总共会经历四个状态，状态流转为 M1 -> M2 -> M3 -> M4，其中 M1 和 M2 是 Pair-verify1 中状态。其中 M1 是接收手机公钥，使用curve算法来生成 车机的私钥和公钥、共享密钥。之后 M1 会被流转到 M2，M2 中会通过算法来生成签名，之后共享密钥加密之后返回给手机。

M1 和 M2 对应着手机第一次 Pair-verify 请求。

M3 是从 M2 流转而来，到第二次接收到 Pair-verify 时，此次的 POST 请求中会携带着手机签名，M3 状态主要是使用共享密钥进行解密，之后对签名后的密钥进行核对，核对没问题之后从 M3 流转到 M4 上来。M4 状态相对比较简单，即回复下手机第二次 Pair-verify 的状态。

Auth Setup

在 Pair-verify 之后，手机还发送了 /auth-setup 这个 POST 请求，下面看下 auth setup 的过程，这个 post 中车机端干了什么，传输了哪些数据给到手机了。

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	fe80::4d4e6ca:29b4:e04c	fe80::2cab:f9ff:fe63:c276	TCP	76	49224 -> 7000 [SYN] Seq=0 Win=65535 Len=0
2	0.001000	fe80::2cab:f9ff:fe63:c276	fe80::4d4e6ca:29b4:e04c	TCP	76	7000 -> 49224 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0
3	0.002000	fe80::4d4e6ca:29b4:e04c	fe80::2cab:f9ff:fe63:c276	TCP	76	49224 -> 7000 [ACK] Seq=1 Ack=1 Win=65535 Len=0
4	0.003000	fe80::4d4e6ca:29b4:e04c	fe80::2cab:f9ff:fe63:c276	RTSP	352	Continuation
5	0.004000	fe80::2cab:f9ff:fe63:c276	fe80::4d4e6ca:29b4:e04c	RTSP	348	Reply: RTSP/1.0 200 OK
6	0.005000	fe80::4d4e6ca:29b4:e04c	fe80::2cab:f9ff:fe63:c276	RTSP	441	Continuation
7	0.006000	fe80::2cab:f9ff:fe63:c276	fe80::4d4e6ca:29b4:e04c	RTSP	108	Reply: RTSP/1.0 200 OK
8	0.007000	fe80::4d4e6ca:29b4:e04c	fe80::2cab:f9ff:fe63:c276	RTSP	236	Continuation
9	0.008000	fe80::2cab:f9ff:fe63:c276	fe80::4d4e6ca:29b4:e04c	RTSP	94	Reply: RTSP/1.0 200 OK
10	0.009000	fe80::4d4e6ca:29b4:e04c	fe80::2cab:f9ff:fe63:c276	RTSP	1056	SETUP rtsp://fe80::2cab:f9ff:fe63:c276/9184174890847455056 RTSP/1.0
11	0.010000	fe80::2cab:f9ff:fe63:c276	fe80::4d4e6ca:29b4:e04c	RTSP	470	Reply: RTSP/1.0 200 OK
12	0.011000	fe80::4d4e6ca:29b4:e04c	fe80::2cab:f9ff:fe63:c276	RTSP	550	Continuation
13	0.012000	fe80::2cab:f9ff:fe63:c276	fe80::4d4e6ca:29b4:e04c	RTSP	27060	Reply: RTSP/1.0 200 OK
14	0.013000	fe80::4d4e6ca:29b4:e04c	fe80::2cab:f9ff:fe63:c276	RTSP	186	RECORD rtsp://fe80::2cab:f9ff:fe63:c276/9184174890847455056 RTSP/1.0
15	0.014000	fe80::2cab:f9ff:fe63:c276	fe80::4d4e6ca:29b4:e04c	RTSP	128	Reply: RTSP/1.0 200 OK
16	0.015000	fe80::4d4e6ca:29b4:e04c	fe80::2cab:f9ff:fe63:c276	RTSP	583	SETUP rtsp://fe80::2cab:f9ff:fe63:c276/9184174890847455056 RTSP/1.0
17	0.016000	fe80::2cab:f9ff:fe63:c276	fe80::4d4e6ca:29b4:e04c	RTSP	297	Reply: RTSP/1.0 200 OK
18	0.017000	fe80::4d4e6ca:29b4:e04c	fe80::2cab:f9ff:fe63:c276	RTSP	411	SET PARAMETER rtsp://fe80::2cab:f9ff:fe63:c276/9184174890847455056 RTSP/1.0
19	0.018000	fe80::2cab:f9ff:fe63:c276	fe80::4d4e6ca:29b4:e04c	RTSP	128	Reply: RTSP/1.0 200 OK
20	0.019000	fe80::4d4e6ca:29b4:e04c	fe80::2cab:f9ff:fe63:c276	RTSP	496	Continuation
21	0.020000	fe80::2cab:f9ff:fe63:c276	fe80::4d4e6ca:29b4:e04c	RTSP	238	Reply: RTSP/1.0 200 OK
22	0.021000	fe80::4d4e6ca:29b4:e04c	fe80::2cab:f9ff:fe63:c276	RTSP	351	Continuation

Frame 8: Packet, 236 bytes on wire (1888 bits), 236 bytes captured (1888 bits) on interface 0
Linux cooked capture v1
Internet Protocol Version 6, Src: fe80::4d4e6ca:29b4:e04c, Dst: fe80::2cab:f9ff:fe63:c276
Transmission Control Protocol, Src Port: 49224, Dst Port: 7000, Seq: 642, Ack: 307, Len: 160
Real-time Streaming Protocol
POST /auth-setup RTSP/1.0\r\n
Content-type: application/octet-stream
CSeq: 2
User-Agent: AirPlay/950.7.1\r\n\r\n
Data: 0151d42f9a8f959e795d450863784efcccf987319ad9354bdcfca8c786778
[length...]

```
static HTTPStatus _requestProcessAuthSetup(AirPlayReceiverConnectionRef inCnx,
HTTPMessageRef inRequest)
{
    HTTPStatus    status;
    OSStatus      err;
    uint8_t       *outputPtr;
    size_t        outputLen;
    HTTPMessageRef response = inCnx->httpCnx->responseMsg;

    aprs_ulog(kAirPlayPhaseLogLevel, "MFi\n");
    outputPtr = NULL;
    require_action(inRequest->bodyOffset > 0, exit, status =
kHTTPStatus_BadRequest);

    // Let MFi-SAP process the input data and generate output data.

    if (inCnx->MFiSAPDone && inCnx->MFiSAP)
    {
        MFiSAP_Delete(inCnx->MFiSAP);
        inCnx->MFiSAP = NULL;
    }
}
```

```

        inCnx->MFISAPDone = false;
    }
    if (!inCnx->MFISAP)
    {
        err = MFISAP_Create(&inCnx->MFISAP, kMFISAPVersion1);
        require_noerr_action(err, exit, status =
kHTTPStatus_InternalServerError);
    }

    err = MFISAP_Exchange(inCnx->MFISAP, inRequest->bodyPtr, inRequest-
>bodyOffset, &outputPtr, &outputLen, &inCnx->MFISAPDone);
    require_noerr_action(err, exit, status = kHTTPStatus_Forbidden);

    // Send the MFi-SAP output data in the response.

    err = HTTPMessageSetBodyPtr(response, kMIMETYPE_Binary, outputPtr,
outputLen);
    require_noerr_action(err, exit, status = kHTTPStatus_InternalServerError);
    outputPtr = NULL;

    status    = kHTTPStatus_OK;

exit:
    if (outputPtr)
        free(outputPtr);
    return (status);
}

```

可以看到这里，`auth setup` 主要是去判断 `Pair-verify` 之后设备是不是一个合法设备，所以这里使用 `mfi sap` 判断这个是否是一个合法 `CarPlay` 设备。`MFISAP_Exchange` 中会去调用 `_MFISAP_Exchange_ServerM1`

```

static OSStatus
_MFISAP_Exchange_ServerM1(
    MFISAPRef      inRef,
    const uint8_t * inInputPtr,
    size_t          inInputLen,
    uint8_t **      outOutputPtr,
    size_t *        outOutputLen )
{
    OSStatus      err;
    const uint8_t * inputEnd;
    const uint8_t * peerPK;
    uint8_t        ourSK[ kMFISAP_ECDHKeyLen ];
    uint8_t        ourPK[ kMFISAP_ECDHKeyLen ];
    SHA_CTX        sha1Ctx;
    SHA256_CTX     sha256Ctx;
    uint8_t        digest[ SHA256_DIGEST_LENGTH ];
    size_t          digestLen;
    uint8_t *       signaturePtr = NULL;
    size_t          signatureLen;
    uint8_t *       certificatePtr = NULL;
    size_t          certificateLen;
    uint8_t        aeskey[ 20 ]; // Only 16 bytes needed for AES, but 20
bytes needed to store full SHA-1 hash.
}

```

```

uint8_t      aesIV[ 20 ];
uint8_t *    buf;
uint8_t *    dst;
size_t       len;

// Throttle requests to no more than 1 per second and linearly back off to 4
seconds max.

if( ( UpTicks() - gMFISAP_LastTicks ) < UpTicksPerSecond() )
{
    if( gMFISAP_ThrottleCounter < 4 ) ++gMFISAP_ThrottleCounter;
    SleepForUpTicks( gMFISAP_ThrottleCounter * UpTicksPerSecond() );
}
else
{
    gMFISAP_ThrottleCounter = 0;
}
gMFISAP_LastTicks = UpTicks();

// Validate inputs. Input data must be: <1:version> <32:client's ECDH public
key>.

inputEnd = inInputPtr + inInputLen;
require_action( inputEnd > inInputPtr, exit, err = kSizeErr ); // Detect bad
length causing ptr wrap.

require_action( ( inputEnd - inInputPtr ) >= kMFISAP_VersionLen, exit, err =
kSizeErr );
inRef->version = *inInputPtr++;
require_action( inRef->version == kMFISAPVersion1, exit, err = kVersionErr );

require_action( ( inputEnd - inInputPtr ) >= kMFISAP_ECDHKeyLen, exit, err =
kSizeErr );
peerPK = inInputPtr;
inInputPtr += kMFISAP_ECDHKeyLen;

require_action( inInputPtr == inputEnd, exit, err = kSizeErr );

// Generate a random ECDH key pair.

err = RandomBytes( ourSK, sizeof( ourSK ) );
require_noerr( err, exit );

//生成公钥。
curve25519( ourPK, ourSK, NULL );

// Use our private key and the client's public key to generate the shared
secret.
// Hash the shared secret with salt and truncate to form the AES key and IV.

//生成共享密钥。
curve25519( inRef->sharedSecret, ourSK, peerPK );

SHA1_Init( &sha1Ctx );
SHA1_Update( &sha1Ctx, kMFISAP_AES_KEY_SaltPtr, kMFISAP_AES_KEY_SaltLen );
SHA1_Update( &sha1Ctx, inRef->sharedSecret, sizeof( inRef->sharedSecret ) );

```

```

SHA1_Final( aesKey, &sha1Ctx );

SHA1_Init( &sha1Ctx );
SHA1_Update( &sha1Ctx, kmfiSAP_AES_IV_SaltPtr, kmfiSAP_AES_IV_SaltLen );
SHA1_Update( &sha1Ctx, inRef->sharedSecret, sizeof( inRef->sharedSecret ) );
SHA1_Final( aesIV, &sha1Ctx );

//获取到 证书信息 Certificate
err = MFiPlatform_CopyCertificate( &certificatePtr, &certificateLen );
require_noerr( err, exit );

// Use the auth chip to sign a hash of <32:our ECDH public key> <32:client's
ECDH public key>.
// And copy the auth chip's certificate so the client can verify the
signature.

if( certificateLen <= 640 )
{
    SHA256_Init( &sha256Ctx );
    SHA256_Update( &sha256Ctx, ourPK, sizeof( ourPK ) );
    SHA256_Update( &sha256Ctx, peerPK, kmfiSAP_ECDHKeyLen );
    SHA256_Final( digest, &sha256Ctx );
    digestLen = sizeof( digest );
}
else
{
    SHA1_Init( &sha1Ctx );
    SHA1_Update( &sha1Ctx, ourPK, sizeof( ourPK ) );
    SHA1_Update( &sha1Ctx, peerPK, kmfiSAP_ECDHKeyLen );
    SHA1_Final( digest, &sha1Ctx );
    digestLen = SHA_DIGEST_LENGTH;
}

//使用摘要信息进行i2c签名。
err = MFiPlatform_CreateSignature( digest, digestLen, &signaturePtr,
&signatureLen );
require_noerr( err, exit );

// Encrypt the signature with the AES key and IV.

err = AES_CTR_Init( &inRef->aesCtx, aesKey, aesIV );
require_noerr( err, exit );
err = AES_CTR_Update( &inRef->aesCtx, signaturePtr, signatureLen,
signaturePtr );
if( err ) AES_CTR_Final( &inRef->aesCtx );
require_noerr( err, exit );
inRef->aesValid = true;

// Return the response:
//
//      <32:our ECDH public key>
//      <4:big endian certificate length>
//      <N:certificate data>
//      <4:big endian signature length>
//      <N:encrypted signature data>

```

```

len = kMFiSAP_ECDHKeyLen + 4 + certificateLen + 4 + signatureLen;
buf = (uint8_t *) malloc( len );
require_action( buf, exit, err = kNoMemoryErr );
dst = buf;
memcpy( dst, ourPK, sizeof( ourPK ) );          dst += sizeof( ourPK );
WriteBig32( dst, certificateLen );                dst += 4;
memcpy( dst, certificatePtr, certificateLen );    dst += certificateLen;
WriteBig32( dst, signatureLen );                  dst += 4;
memcpy( dst, signaturePtr, signatureLen );        dst += signatureLen;

check( dst == ( buf + len ) );
*outOutputPtr = buf;
*outOutputLen = (size_t)( dst - buf );

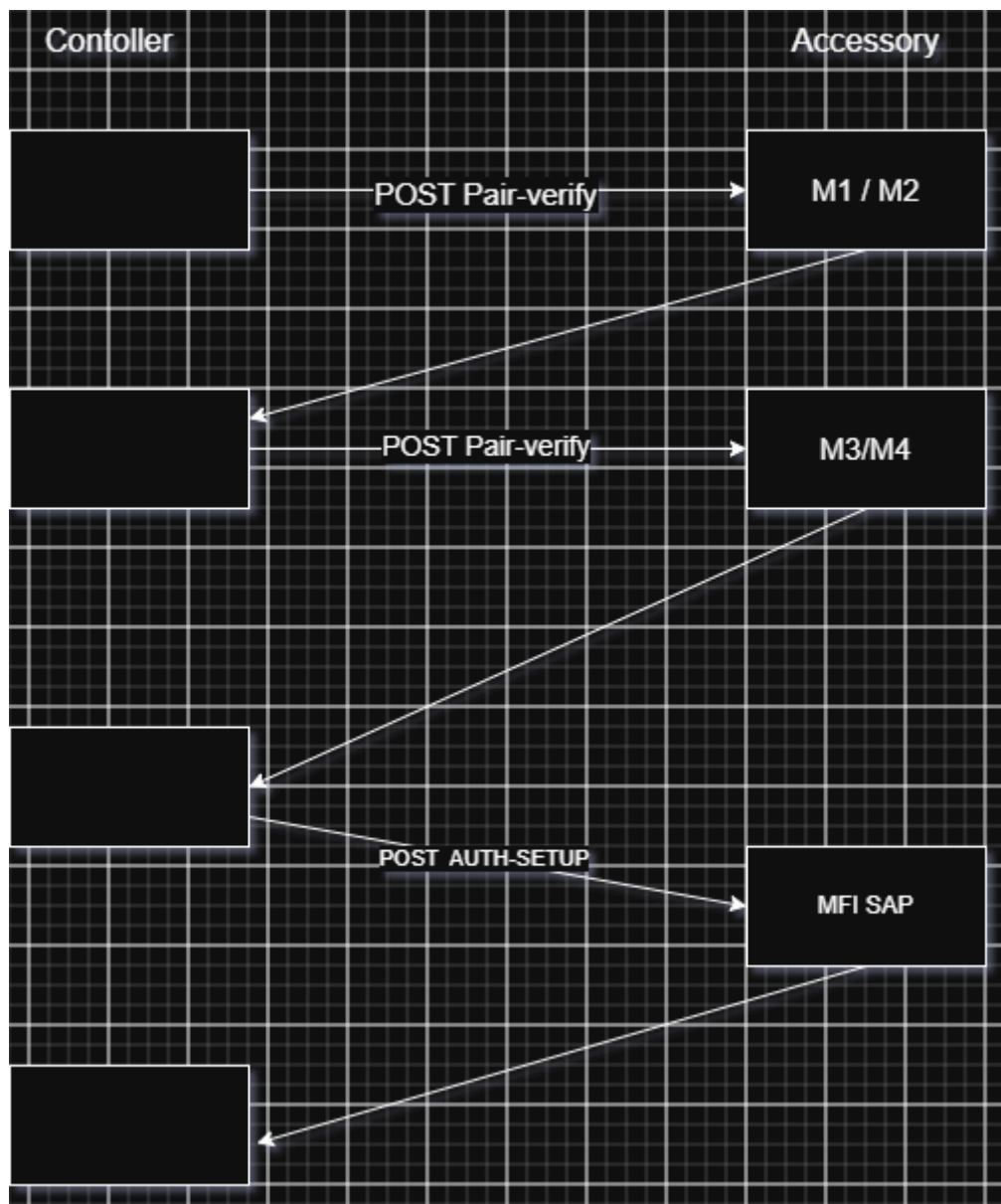
exit:
FreeNullSafe( certificatePtr );
FreeNullSafe( signaturePtr );
return( err );
}

```

`_MFiSAP_Exchange_ServerM1` 中会去重新 公钥和私钥、共享密钥。之后获取到 `i2c` 的证书信息，之后将共享密钥获取签名。最终将相对应的信息都返回给手机了。

经过 `Pair-verify` 和 `Auth Setup` 之后，手机可以判断连接车机是一个合法 `CarPlay` 设备。

整体流程为下面图所示：



经过 **Pair-verify** 和 **Auth-setup** 之后，手机可以认为车机是一个可靠 **CarPlay** 设备，并且后续通信方式都是使用加密通信了。